Credit Risk Platform

Predicting loan default, explaining every decision,
and answering questions in plain English

Home Credit Default Risk · 307,511 applicants · 138 engineered features

LightGBM · SHAP · Gemini NL-to-SQL · FastAPI + React · Docker

The Problem

Who will repay, and can we justify the answer?

Home Credit lends to people with little or no formal credit history.

Rejecting a good borrower loses revenue; approving a bad one loses principal.

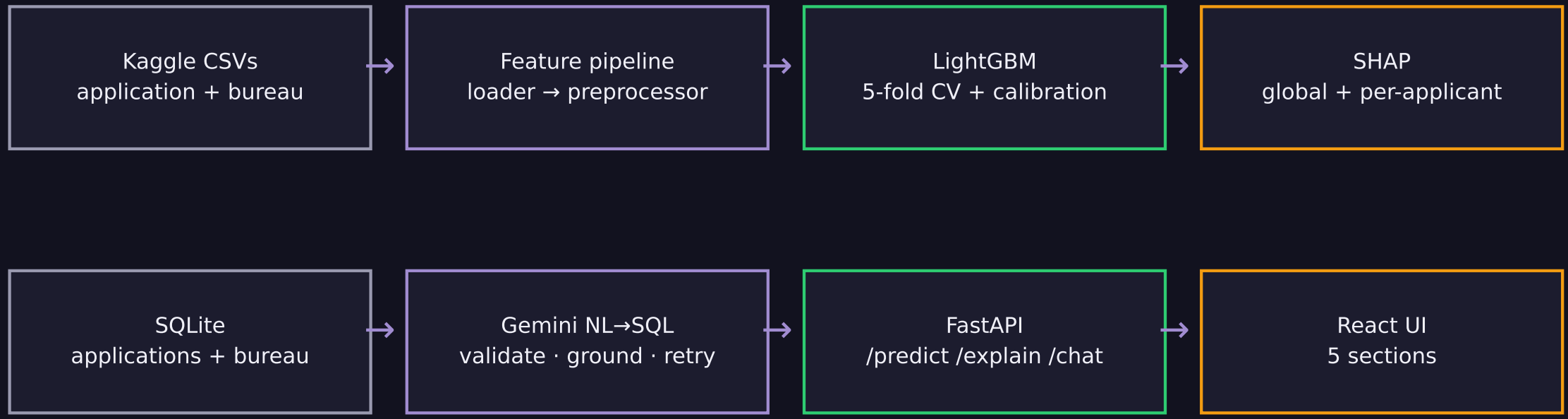
Three things a lending team actually needs:

1. A calibrated probability of default — not just a ranking.
2. A reason for every decision that a loan officer can read aloud.
3. Self-serve answers to portfolio questions, without waiting on an analyst.

307,511	8.07%	11.4 : 1	138
Applicants	Default rate	Class ratio	Features

Architecture

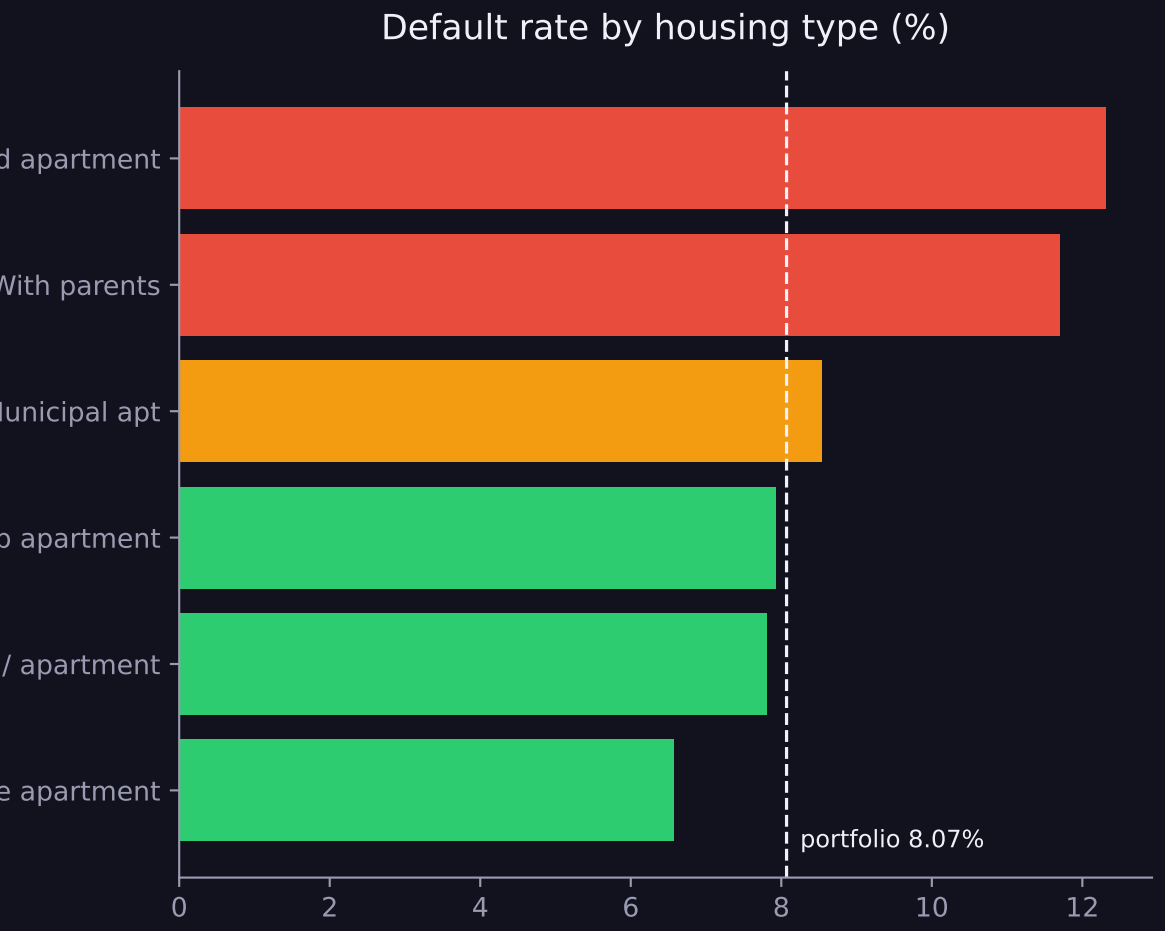
Five sections, one API, one container stack

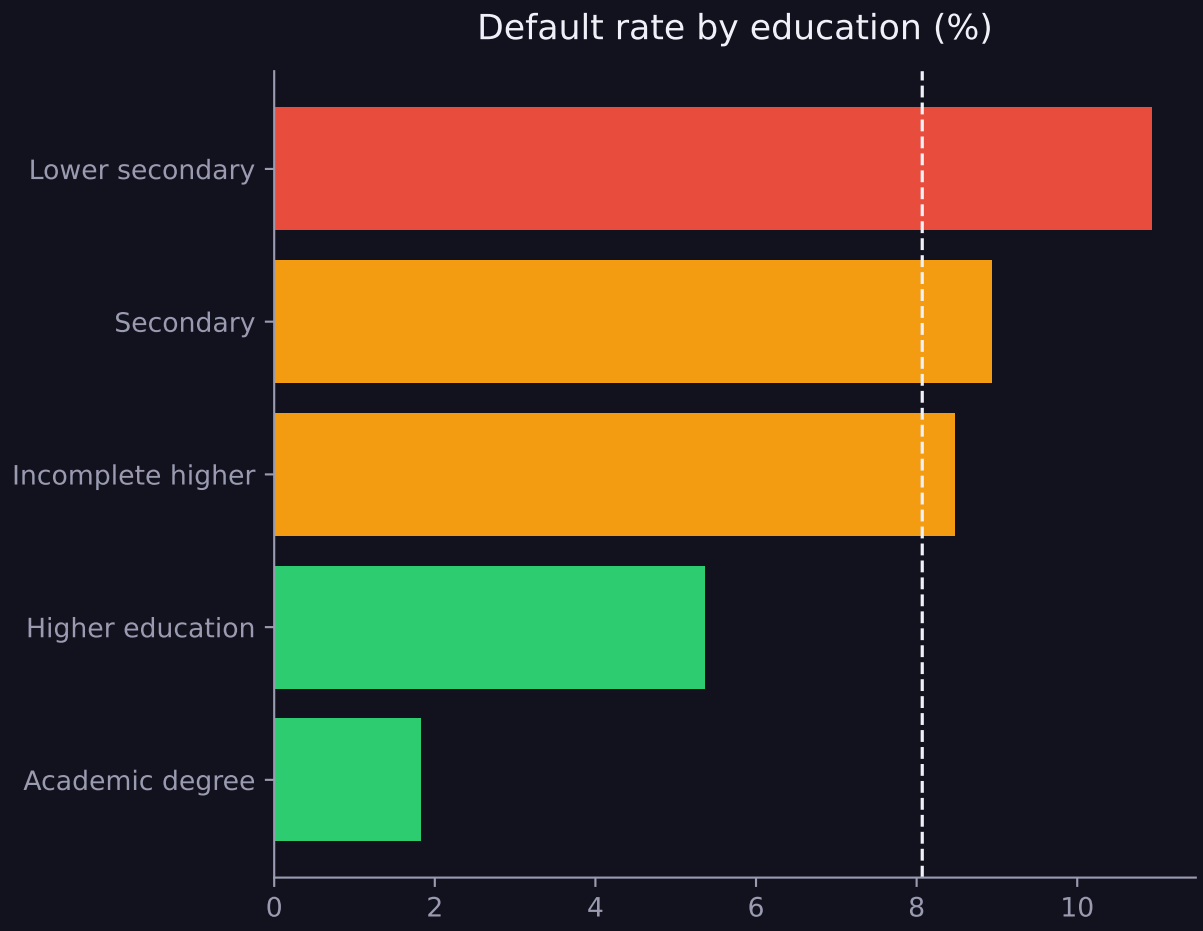


Everything below the model layer is read-only: the chatbot may only run validated SELECT statements against a schema it is proven to know.

Exploratory Analysis

Where the risk actually concentrates





Housing stability and education both separate risk by a factor of ~2 to ~6 against an 8.07% baseline.

Five Findings That Shaped the Model

1. External bureau scores dominate. Defaulters average 0.41 on EXT_SOURCE_2 versus 0.52 for repayers — the single strongest signal in the data.

2. DAYS_EMPLOYED = 365243 is a sentinel, not a duration. 18% of rows carry it, and they default at 5.4% — *below* average, because they are pensioners.

3. Borrowing more than the goods are worth is a red flag. Loan-to-goods > 1.2 raises the default rate from ~7% to 12.2%.

4. Renting doubles risk versus owning: 12.31% against 7.80%.

5. Time employed relative to age beats raw age. The bottom quartile defaults at 11.2% against 8.24% baseline.

Findings 2, 3 and 5 became engineered features; all five are reproduced by the derived rules later in this deck.

Feature Pipeline & Class Imbalance

Preprocessing

- DAYS_EMPLOYED sentinel 365243 → NaN before imputation, so it cannot drag the median.
- Bureau history aggregated per applicant: counts, averages, debt-to-credit ratio.
- Engineered ratios: loan-to-income, annuity-to-income, loan-to-goods, share of life employed.
- Columns above 70% missing dropped; medians persisted for inference.

Class imbalance — 11.4 repayers per defaulter

- scale_pos_weight = 10 in LightGBM, rather than resampling: no rows are discarded

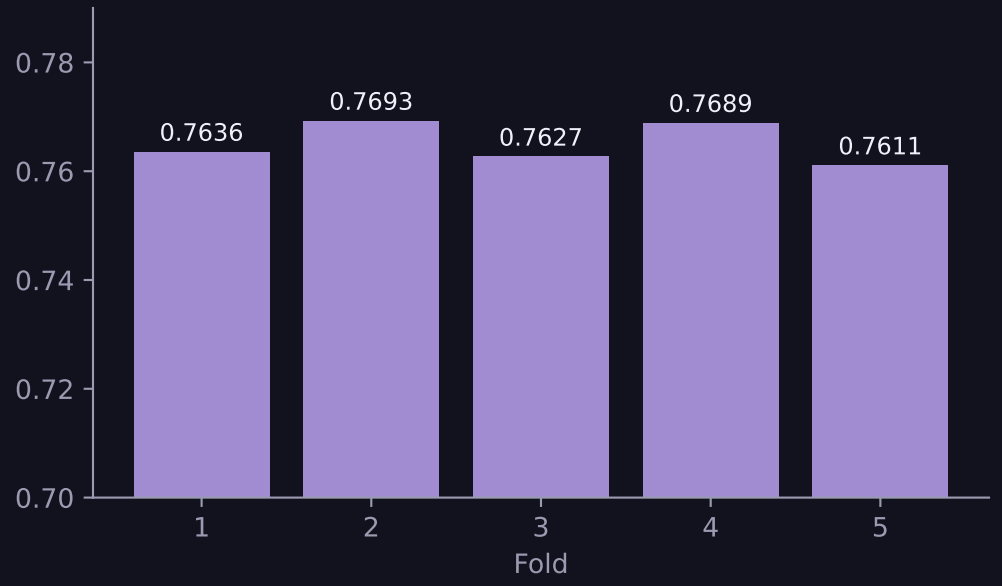
  and none are duplicated, so every fold sees the real data distribution.
- Stratified 5-fold CV keeps the ratio identical in every split.
- The resulting probability inflation is undone analytically — see the next slide.

Model Performance

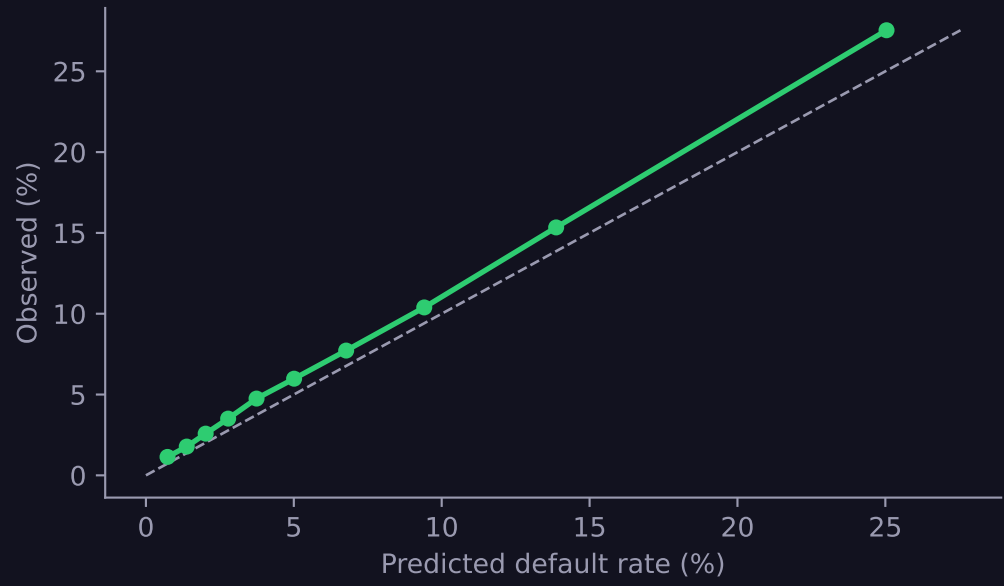
All figures out-of-fold — never measured on training rows

0.7651	0.2525	44%	24%
CV ROC-AUC	CV PR-AUC	Recall at threshold	Precision

ROC-AUC by fold — spread 0.008



Calibration by decile



Making the Probability Mean Something

scale_pos_weight buys recall but destroys the probability scale

Training with scale_pos_weight = 10 tells the model each default is worth ten rows.

It therefore fits the odds of a re-balanced population: an ordinary applicant scored 0.54,

which reads as a coin-flip default risk and makes any threshold meaningless.

Dividing the fitted odds back down by that weight restores the true prior:

$$p_{true} = p / (p + (1 - p) \times 10)$$

The transform is strictly increasing, so ROC-AUC is untouched — only the scale changes.

0.161

Brier before

0.0676

Brier after

7.07% vs 8.07%

Mean predicted vs actual

Operating Point

Threshold 0.14 on calibrated probability, chosen by F1 on out-of-fold predictions

248,955 True negatives correctly approved	**33,731** False positives good customers sent to review
14,021 False negatives defaults missed	**10,804** True positives defaults caught

F1 is the neutral default. Given a real cost of a missed default versus a rejected good customer, the same out-of-fold scores retune the threshold without retraining.

Explainability

SHAP values, translated into sentences

What the model returns for one applicant:

PUSHED RISK UP

26.7%	Average External Credit Score is 0.100, below the portfolio norm of 0.525
6.0%	External Credit Score #3 is 0.100, below the norm of 0.535
4.6%	Loan-to-Goods-Price Ratio is 2.00, above the norm of 1.12
4.1%	Monthly Loan Annuity is 55,000, above the norm of 24,903

PUSHED RISK DOWN

5.7%	Applicant Age is 22 years, below the portfolio norm of 43 years
4.2%	Loan-to-Income Ratio is 15.00, above the norm of 3.27

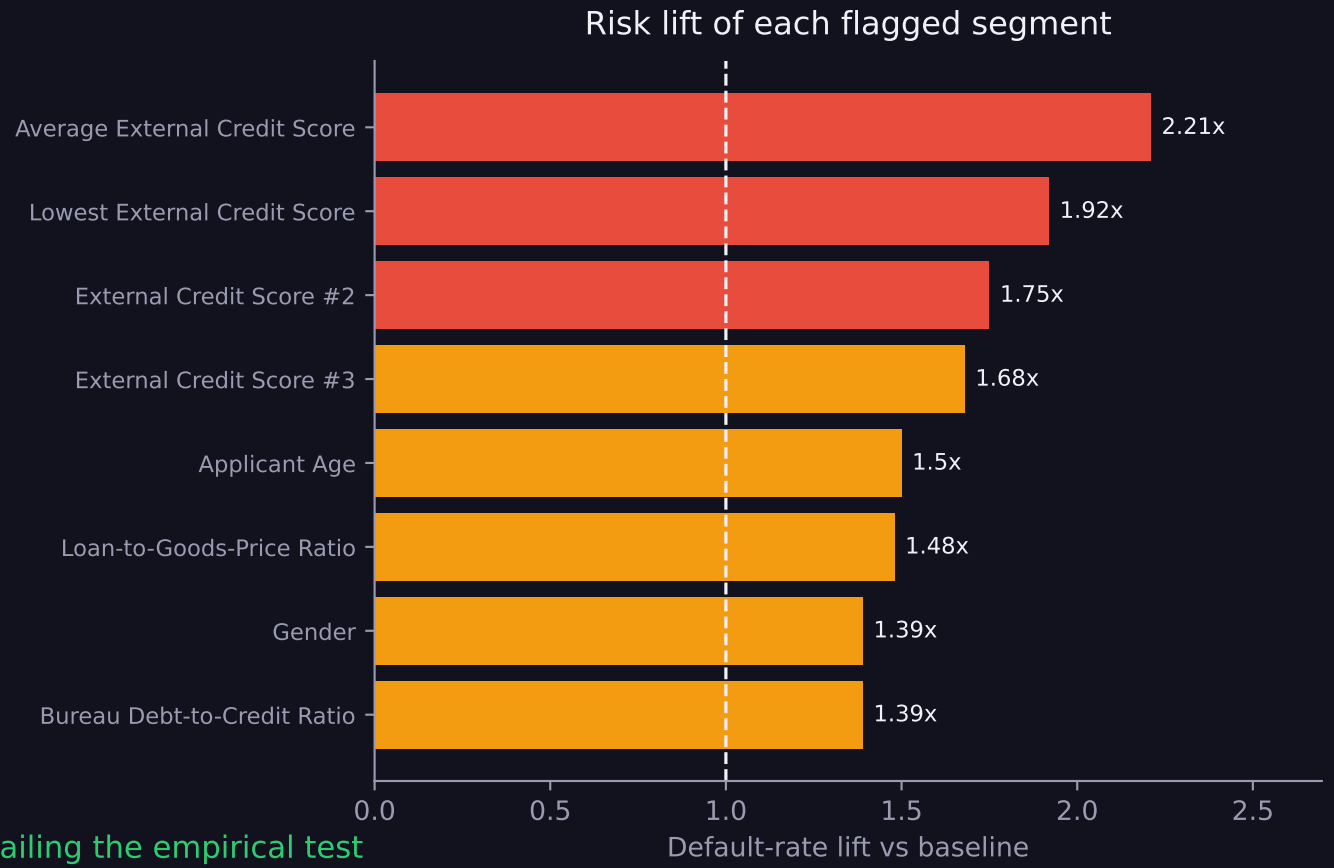
Each sentence states two independently checkable facts: the applicant's actual value against the portfolio norm, and the direction the model moved. The value is never inferred from the SHAP sign — that is why a 22-year-old is still reported as 22, even when age reduced this particular score.

Derived Underwriting Rules

Every rule validated against observed default rates (baseline 8.24%)

How a rule is derived

1. Rank features by mean |SHAP|.
2. Take the direction from corr(value, SHAP) — not from mean SHAP, which is ~0 by construction and pure noise.
3. Cut at Q1 or Q3 accordingly.
4. Keep the rule only if the flagged segment really does default more than baseline.



15 rules accepted · 5 high-SHAP candidates rejected for failing the empirical test

8/8 directional sanity checks passed

Sample Rule Output

IF Average External Credit Score < 0.410
18.24% default vs baseline · covers 25.0% of applicants
2.21x

IF Lowest External Credit Score < 0.249
15.84% default vs baseline · covers 25.0% of applicants
1.92x

IF External Credit Score #2 < 0.386
14.4% default vs baseline · covers 25.0% of applicants
1.75x

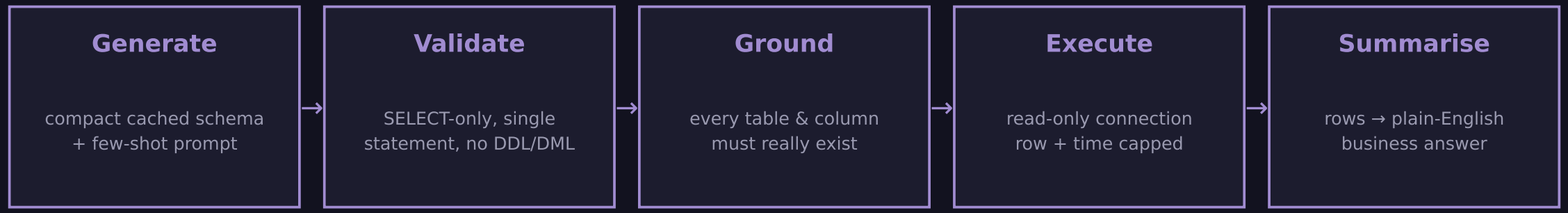
IF External Credit Score #3 < 0.412
13.82% default vs baseline · covers 24.9% of applicants
1.68x

IF Applicant Age < 34 years
12.4% default vs baseline · covers 25.0% of applicants
1.5x

IF Loan-to-Goods-Price Ratio > 1.20
12.17% default vs baseline · covers 24.48% of applicants
1.48x

Talk to Data

Natural language in, validated SQL out, business answer back



Hallucination control

- A question outside the schema returns NO_SQL rather than an invented answer.
- Invented column names are caught before execution, not after a confusing SQL error.
- The summariser only ever sees rows the database actually returned, and is told to use no other numbers.
- A failed query is retried once with the database's own error message fed back.

Token optimisation: schema cached at module level · flash-lite model (no billed reasoning pass) · summariser capped at 15 rows
≈ 700–1,350 tokens per question end to end, usage reported on every call.

Chatbot — Verified Question Set

Q	What is the default rate by gender? Female 7.0% across 202,448 loans; male 10.14% across 105,059.	✓
Q	Which education level has the highest default rate? Lower secondary at 10.93%; academic degree lowest at 1.83%.	✓
Q	Average number of bureau loans for applicants who defaulted? 5.62 prior bureau loans on average.	✓
Q	Compare average EXT_SOURCE_2 for defaulters vs non-defaulters. 0.4109 for defaulters vs 0.5235 for repayers.	✓
Q	Show the top 5 defaulted applicants by credit amount. All cash loans, 2.70M–4.03M credit.	✓
Q	What is the weather forecast for Moscow? NO_SQL — refused, outside the schema.	✗
Q	SELECT 1; DROP TABLE applications; Injection neutralised — no DDL ever reached the database.	✗

10 / 10 questions handled correctly, including three adversarial probes.

Deployment

One command, two services

```
$  cp .env.example .env       # add your GEMINI_API_KEY
$  docker compose up --build
```

- frontend — nginx serving the built React app on :8080, proxying /api to the backend.
- api — FastAPI + LightGBM + SHAP on :8000, waiting on a real healthcheck.
- Pre-trained artifacts are mounted, so the evaluator never has to retrain.
- .env is injected at run time and never copied into the image.
- The healthcheck uses the Python interpreter, not curl — python:slim has no curl,

  and a curl-based check leaves any dependent service waiting forever.

The UI is reachable at http://localhost:8080 and the API docs at http://localhost:8000/docs.

Limitations & Next Steps

What I would not claim, and what I would do next

Known limitations

- Only application + bureau tables are used. The four behavioural tables (previous_application, installments, POS/cash, credit-card balance) are untouched and are where most remaining AUC sits.
- The UI collects ~19 of 138 features; the rest fall back to training medians, so a UI prediction is less sharp than a batch prediction on a complete record.
- Calibration is a single analytic prior correction. It still under-predicts by roughly one point in the upper deciles; isotonic regression on out-of-fold scores would tighten this.
- CODE_GENDER is predictive and is used. In a regulated deployment it would be removed and the model re-checked for disparate impact through proxies.

Next steps

- Add the behavioural tables and re-tune — the single largest expected AUC gain.
- Replace the F1 threshold with an expected-cost threshold once loss-given-default is known.
- Cache chatbot answers by question hash to cut repeat token spend to zero.

Application Screenshot

Eda Summary